例 3.3 画图说明用异步串行传输方式发送十六进制数据 95H。要求字符格式为:1 位起始位、8 位数据位、1 位偶校验位、1 位终止位。

解:异步串行传送在起始位之后传输的是数据位的最低位(95H 的最低位 $D_0=1$),而且数据位的最高位(95H 的最高位 $D_7=1$)传输之后传输校验位,最后是终止位。数据 95H 的偶校验位为 0,其波形图如图 3.20 所示。

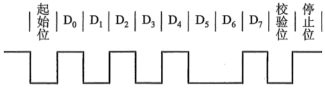


图 3.20 数据 95H 的传送波形

异步串行通信的数据传送速率用波特率来衡量。波特率是指单位时间内传送二进制数据的位数,单位用 bps(位/秒)表示,也作波特。

例 3.2 在异步串行传输系统中,假设每秒传输 120 个数据帧,其字符格式规定包含 1 个起始位、7 个数据位、1 个奇校验位、1 个终止位,试计算波特率。

解:根据题目给出的字符格式,一帧包含 1+7+1+1=10 位

故波特率为(1+7+1+1)×120=1 200 bps=1 200 波特

由于异步串行通信字符格式中包含若干附加位,如起始位、终止位、校验位,而且终止位又有 1 位、1.5 位、2 位之分,若只考虑有效数据位,可用比特率来衡量异步串行通信的数据传输速率,即单位时间内传送二进制有效数据的位数,单位用 bps 表示。

例 3.4 在异步串行传输系统中,若字符格式为:1 位起始位、8 位数据位、1 位奇校验位、1 位终止位。假设波特率为 1 200 bps,求这时的比特率。

解:根据题目给出的字符格式,有效数据位为 8 位,而传送一个字符需 1+8+1+1=11 位,故比特率为

$$1\ 200\times(8/11)=872.72\ \text{bps}$$

(3) 带 Cache 的 DRAM (CDRAM)

带 Cache 的 DRAM 是在通常的 DRAM 芯片内又集成了一个小的 SRAM,又称增强型的 DRAM(EDRAM)。图 4.48 是 1 M×4 位的 CDRAM,其中 SRAM 为 512×4 位,DRAM 排列成 2048×512×4 位的阵列。

从图 4.48 所示的结构可见,芯片内的数据输出路径(由 SRAM 到 I/O)与数据输入路径(由 I/O 到读放大器和列写选择)是分开的,这就允许在写操作完成的同时启动同一行的读操作。此外,在 SRAM 读出期间可同时对 DRAM 阵列进行刷新。

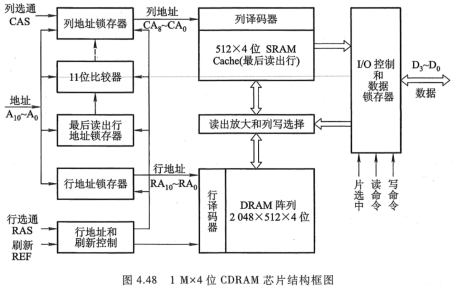


图 4.48 1 M×4 位 CDRAM 芯片结构框图

当然,以较小的硬件代价使 Cache-主存系统的平均访问时间 t_a 越接近于 t_c 越好。用 e 表示访问效率,则有

$$e=\frac{t_c}{t_a}\times100\%=\frac{t_c}{ht_c+(1-h)t_m}\times100\%$$

可见,为提高访问效率,命中率 h 越接近 1 越好。

(2) 由题可知,访问主存的时间是访问 Cache 时间的 4 倍(200/50=4)。

设访问 Cache 的时间为 t,访问主存的时间为 $4t$,Cache-主存系统的访问效率为 e,则

$$e=\frac{\text{访问 Cache 的时间}}{\text{平均访问时间}}\times100\%$$

$$=\frac{t}{0.97\times t+(1-0.97)\times4t}\times100\%=91.7\%$$

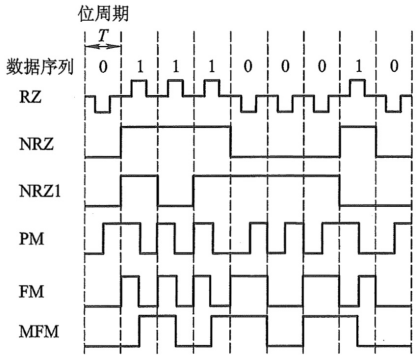


图 4.63　六种磁记录方式的写入电流波形

（1）归零制（RZ）

归零制记录“1”时，通以正向脉冲电流，记录“0”时，通以反向脉冲电流。这样使其在磁表面形成两个不同极性的磁饱和状态，分别表示“1”和“0”。由于两位信息之间驱动电流归零，故称为归零制记录方式。这种方式在写入信息时很难覆盖原来的磁化区域，所以为了重新写入信息，在写入前，必须先抹去原存信息。这种记录方式原理简单，实施方便，但由于两个脉冲之间有一段间隔没有电流，相应的该段磁介质未被磁化，即该段空白，故记录密度不高，目前很少使用。

（2）不归零制（NRZ）

不归零制记录信息时，磁头线圈始终有驱动电流，不是正向，便是反向，不存在无电流状态。这样，磁表面层不是正向被磁化，就是反向被磁化。当连续记录“1”或“0”时，其写电流方向不变，只有当相邻两信息代码不同时，写电流才改变方向，故称为“见变就翻”的不归零制。

（3）“见 1 就翻”的不归零制（NRZ1）

“见 1 就翻”的不归零制在记录信息时，磁头线圈也始终有电流。但只有在记录“1”时电流改变方向，使磁层磁化方向发生翻转；记录“0”时，电流方向保持不变，使磁层的磁化方向也维持原来状态，因此称为“见 1 就翻”的不归零制。

（4）调相制（PM）

调相制又称为相位编码（PE），其特点是记录“1”或“0”的相位相反。如：记录“0”时，写电流由负变正；记录“1”时，写电流由正变负（也可以相反定义），而且电流变化出现在一位信息记录时间的中间时刻，它以相位差为 180°的磁化翻转方向来表示“1”和“0”。因此，当连续记录相同信息时，在每两个相同信息的交界处，电流方向都要变化一次；若相邻信息不同，则两个信息位的交界处电流方向维持不变。调相制在磁带存储器中用得较多。

（5）调频制（FM）

调频制的记录规则是：以驱动电流变化的频率不同来区别记录“1”还是“0”。当记录“0”时，在一位信息的记录时间内电流保持不变；当记录“1”时，在一位信息记录时间的中间时刻，使电流改变一次方向。而且无论记录“0”还是“1”，在相邻信息的交界处，线圈电流均变化一次。因此，写“1”时，在位单元的起始和中间位置都有磁通翻转；在写“0”时，仅在位单元起始位置有翻转。显然，记录“1”的磁翻转频率为记录“0”的两倍，故又称为倍频制。调频制记录方式被广泛应用在硬磁盘和软磁盘中。

（6）改进型调频制（MFM）

这种记录方式基本上同调频制，即记录“0”时，在位记录时间内电流不变；记录“1”时，在位记录时间的中间时刻电流发生一次变化。两者不同之处在于，改进型调频制只有当连续记录两个或两个以上的“0”时，才在每位的起始处改变一次电流，不必在每个位起始处都改变电流方向。由于这一特点，在写入同样数据序列时，MFM 比 FM 磁翻转次数少，在相同长度的磁层上可记录的信息量将会增加，从而提高了磁记录密度。FM 制记录一位二进制代码最多是两次磁翻转，MFM 制最多只要一次翻转，记录密度提高了一倍，故又称为倍密度记录方式。倍密度软磁盘即采用 MFM 记录方式。

此外还有一种二次改进的调频制（M^2FM），它是在 MFM 基础上改进的，其记录规则是：当连续记录“0”时，仅在第 1 个位起始处改变电流方向，以后的位交界处电流方向不变。

相联存储器的每个字由若干字段组成，每个字段描述了一个对象的属性，也称一个内容。例如，在存储学生信息的相联存储器中，可分为学号、姓名、年龄、班号、成绩等字段（参见图 4.82）。

相联存储器的基本组成如图 4.81 所示。

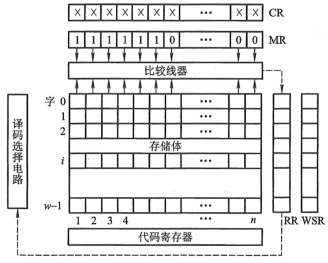


图 4.81　相联存储器基本组成框图

图中检索寄存器 CR 用来存放检索字，其位数与相联存储器的字长相等。屏蔽寄存器 MR 用来存放屏蔽码，其位数与检索寄存器位数相同，其内容与需要检索的字段有关。如需检索 CR 的高 6 位字段（称为检索项），则 MR 的高 6 位为“1”，其余各位为“0”，即把 CR 中的第 7～n 位屏蔽掉，也即这些位不参加比较。比较线路是把检索项和所有存储单元的相应位进行比较，如果比较结果相等，就将符合寄存器 RR 的相应位置“1”。RR 又称为查找结果寄存器，其位数等于相联存储器的字数。如果比较结果第 i 个字满足要求，则 RR 的第 i 位为“1”，其余各位为“0”；如果同时有 5 个字都满足要求，则 RR 中就有 5 位为“1”。有的相联存储器还设有字选择寄存器 WSR，用来确定哪些存储字参与检索。若 WSR 某位为“1”，则表示对应的存储字参与检索，而对应 WSR 某位为“0”的存储字则不参与检索。可见 WSR 的位数与存储器字数相同。代码寄存器用来存放从存储体中读出的代码，或存放写至存储体中的代码。

方式的主要指标

录方式的优劣标准主要反映在编码效率和自同步能力等方面。

例 5.3 一个 DMA 接口可采用周期窃取方式把字符传送到存储器，它支持的最大批量为 400 个字节。若存取周期为 100 ns，每处理一次中断需 5 μs，现有的字符设备的传输率为 9 600 bps。假设字符之间的传输是无间隙的，若忽略预处理所需的时间，试问采用 DMA 方式每秒因数据传输需占用处理器多少时间？如果完全采用中断方式，又需占用处理器多少时间？

解：根据字符设备的传输率为 9 600 bps，则每秒能传输

$$9\,600/8 = 1\,200\ \text{B}(1\,200\ \text{个字符})$$

若采用 DMA 方式，传送 1 200 个字符共需 1 200 个存取周期，考虑到每传 400 个字符需中断处理一次，因此 DMA 方式每秒因数据传输占用处理器的时间是

$$0.1\ \mu s \times 1\,200 + 5\ \mu s \times (1\,200 / 400) = 135\ \mu s$$

若采用中断方式，每传送一个字符要申请一次中断请求，每秒因数据传输占用处理器的时间是

$$5\ \mu s \times 1\,200 = 6\,000\ \mu s$$

如果采用双符号位的补码，则

当 $S>0$ 时，其补码规格化形式为

$$[S]_{补} = 00.1\times\times\cdots\times \tag{6.20}$$

当 $S<0$ 时，其补码规格化形式为

$$[S]_{补} = 11.0\times\times\cdots\times \tag{6.21}$$

可见，当尾数的最高数值位与符号位不同时，即为规格化形式，但对 $S<0$ 时，有两种情况需特殊处理。

3. 规格化

由 6.2.2 节可知，当基值 $r=2$ 时，尾数 S 的规格化形式为

$$\frac{1}{2} \leqslant |S| < 1 \tag{6.19}$$

① $S=-\frac{1}{2}$，则 $[S]_{补}=11.100\cdots0$。此时对于真值 $-\frac{1}{2}$ 而言，它满足式(6.19)，对于补码($[S]_{补}$)而言，它不满足于式(6.21)。为了便于硬件判断，特规定 $-\frac{1}{2}$ 不是规格化的数(对补码而言)。

② $S=-1$，则 $[S]_{补}=11.00\cdots0$，因小数补码允许表示 -1，故 -1 视为规格化的数。

还有一种条件转移指令，SKP(Skip)，它暗示其下一条指令将被跳过，从而隐含了转移地址是 SKP 后的第二条指令。例如：

```
200
 ⋮
205 SKP DZ
206
207
```

这里“SKP DZ”表示若设备的完成触发器 D 为零，则执行完 205 条指令后，立即跳至第 207 条指令，再顺序执行。

这种寻址方式与直接寻址相比，它扩大了操作数的寻址范围，因为 A 的位数通常小于指令字长，而存储字长可与指令字长相等。若设指令字长和存储字长均为 16 位，A 为 8 位，显然直接寻址范围为 2^8，一次间接寻址的寻址范围可达 2^{16}。当多次间接寻址时，可用存储字的首位来标志间接寻址是否结束。如图 7.11(b)中，当存储字首位为“1”时，标明还需继续访存寻址；当存储字首位为“0”时，标明该存储字即为 EA。由此可见，存储字首位不能作为 EA 的组成部分，因此，它的寻址范围为 2^{15}。

间接寻址的第二个优点在于它便于编制程序。例如，用间接寻址可以很方便地完成子程序返回，图 7.12 示意了用于子程序返回的间址过程。

由于可以在许多处调用子程序,因此,CPU 必须记住返回地址,使子程序能准确返回。返回地址可存放在以下 3 处。

- 寄存器内。机器内设有专用寄存器,专门用于存放返回地址。
- 子程序的入口地址内。
- 栈顶内。现代计算机都设有堆栈,执行 RETURN 指令后,便可自动从栈顶内取出应返回的地址。

例 7.7 设某机共能完成 110 种操作,CPU 有 8 个通用寄存器(16 位),主存容量为 4 M 字,采用寄存器-存储器型指令。

(1) 欲使指令可直接访问主存的任一地址,指令字长应取多少位?画出指令格式。

(2) 若在上述设计的指令字中设置一寻址特征位 X,且 X = 1 表示某个寄存器作基址寄存器,画出指令格式。试问基址寻址可否访问主存的任一单元?为什么?如果不能,提出一种方案,使其可访问主存的任一位置。

(3) 若主存容量扩大到 4 G 字,且存储字长等于指令字长,则在不改变上述硬件结构的前提下,可采用什么方法使指令可访问存储器的任一位置?

解:(1) 欲使指令可直接访问 4 M 字存储器的任一单元,采用寄存器-存储器型指令,该机指令应包括 22 位的地址码、3 位寄存器编号和 7 位操作码,即指令字长取 22+3+7 = 32 位,指令格式如下:

7	3	22
OP	R	A

(2) 在上述指令格式中增设一寻址特征位,且 X = 1 表示某个寄存器作基址寄存器 R_B。其指令格式如下:

7	3	1	3	18
OP	R	X	R_B	A

由于通用寄存器仅 16 位,形式地址 18 位,不足以覆盖 4 M 地址空间,可将 R_B 寄存器内容左移 6 位,低位补 0,形成 22 位基地址,然后与形式地址相加,所得的有效地址即可访问 4 M 字存储器的任一单元。

(3) 若主存容量扩大到 4 G 字,且存储字长等于指令字长,则在不改变上述硬件结构的前提下,采用一次间接寻址即可访问存储器的任一单元,因为间接寻址后得到的有效地址为 32 位,2^{32} = 4 G。

3. 效率(Efficiency)

效率是指流水线中各功能段的利用率。由于流水线有建立时间和排空时间,因此各功能段的设备不可能一直处于工作状态,总有一段空闲时间。图 8.18 是 4 段(m = 4)流水线的时空图,各段时间相等,均为 Δt。图中 $mn\Delta t$ 是流水线各段处于工作时间的时空区,而流水线中各段总的时空区是 $m(m+n-1)\Delta t$。通常用流水线各段处于工作时间的时空区与流水线中各段总的时空区之比来衡量流水线的效率。用公式表示为

$$E=\frac{mn\Delta t}{m(m+n-1)\Delta t}=\frac{n}{m+n-1}=\frac{S_p}{m}=T_p\Delta t$$



图 8.18 各段时间相等的流水线时空图

进一步分析发现，机器内的各种操作大致可归属为对 CPU 内部的操作和对主存的操作两大类，由于 CPU 内部的操作速度较快，CPU 访存的操作时间较长，因此通常以访问一次存储器的时间定为基准时间较为合理，这个基准时间就是机器周期。又由于不论执行什么指令，都需要访问存储器取出指令，因此在存储字长等于指令字长的前提下，取指周期也可看作机器周期。

2. 时钟周期（节拍、状态）

在一个机器周期里可完成若干个微操作，每个微操作都需要一定的时间，可用时钟信号来控制产生每一个微操作命令（如图 9.3 中的 C_i）。时钟就好比计算机的心脏，只要接通电源，计算

机内就会产生时钟信号。时钟信号可由机器主振电路（如晶体振荡器）发出的脉冲信号经整形（或倍频、分频）后产生，时钟信号的频率即为 CPU 主频。用时钟信号控制节拍发生器，就可产生节拍。每个节拍的宽度正好对应一个时钟周期。在每个节拍内机器可完成一个或几个需同时执行的操作，它是控制计算机操作的最小时间单位。图 9.8 反映了机器周期、时钟周期和节拍的关系，图中一个机器周期内有 4 个节拍 T_0、T_1、T_2、T_3。

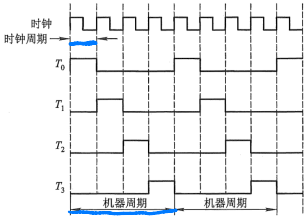


图 9.8　机器周期、时钟周期和节拍的关系

（3）采用中央控制和局部控制相结合的方法

这种方案将机器的大部分指令安排在统一的、较短的机器周期内完成，称为中央控制，而将少数操作复杂的指令中的某些操作（如乘除法和浮点运算等）采用局部控制方式来完成，图 9.11 所示为中央控制和局部控制的时序关系。

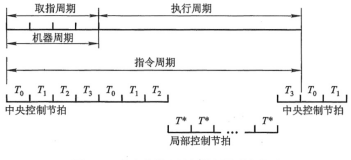


图 9.11　中央控制和局部控制的时序关系

在设计局部控制线路时需要注意两点：其一，使局部控制的每一个节拍 T^* 的宽度与中央控制的节拍宽度相同；其二，将局部控制节拍作为中央控制中机器节拍的延续，插入中央控制的执行周期内，使机器以同样的节奏工作，保证局部控制和中央控制的同步。T^* 的多少可根据情况

而定，对于乘法，当操作数位数固定后，T^* 的个数也就确定了。而对于浮点运算的对阶操作，由于移位次数不是一个固定值，因此 T^* 的个数不能事先确定。

以乘法指令为例，第一个机器周期采用中央控制的节拍控制取指令操作，接着仍用中央控制的 T_0、T_1、T_2 节拍去完成将操作数从存储器中取出并送至寄存器的操作，然后转局部控制，用局部控制节拍 T^* 完成重复加和移位的操作。

控制单元控制一条指令执行的过程实质上是依次执行一个确定的微操作序列的过程。由于不同指令所对应的微操作数及其复杂程度不同，因此每条指令和每个微操作所需的执行时间也不同。通常将如何形成控制不同微操作序列所采用的时序控制方式称为 CU 的控制方式。常见的控制方式有同步控制、异步控制、联合控制和人工控制四种。

1. 同步控制方式

同步控制方式是指，任何一条指令或指令中任何一个微操作的执行都是事先确定的，并且都是受统一基准时标的时序信号所控制的方式。

图 9.9（a）就是一种典型的同步控制方式，每个机器周期都包含 4 个节拍。如果机器内的存

储器存取周期不统一，那么只有把最长的存取周期作为机器周期，才能采用同步控制，否则取指令和取数时间不同，无法用统一的基准。又如有些不访存的指令，执行周期的微操作较少，无须 4 个节拍。因此，为了提高 CPU 的效率，在同步控制中又有三种方案。

（1）采用定长的机器周期

这种方案的特点是：不论指令所对应的微操作序列有多长，也不管微操作的简繁，一律以最长的微操作序列和最繁的微操作作为标准，采取完全统一的、具有相同时间间隔和相同数目的节拍作为机器周期来运行各种不同的指令，如图 9.9（a）所示。显然，这种方案对于微操作序列较短的指令来说，会造成时间上的浪费。

（2）采用不定长的机器周期

采用这种方案时，每个机器周期内的节拍数可以不等，如图 9.9（b）所示。这种控制方式可解决微操作执行时间不统一的问题。通常把大多数微操作安排在一个较短的机器周期内完成，而对某些复杂的微操作，采用延长机器周期或增加节拍的办法来解决，如图 9.10 所示。

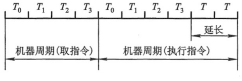


图 9.10　延长机器周期示意

2. 异步控制方式

异步控制方式不存在基准时标信号，没有固定的周期节拍和严格的时钟同步，执行每条指令和每个操作需要多少时间就占用多少时间。这种方式微操作的时序由专门的应答线路控制，即当 CU 发出执行某一微操作的控制信号后，等待执行部件完成该操作后发回“回答”（或“结束”）信号，再开始新的微操作，使 CPU 没有空闲状态，但因需要采用各种应答电路，故其结构比同步控制方式复杂。

3. 联合控制方式

同步控制和异步控制相结合就是联合控制方式。这种方式对各种不同指令的微操作实行大部分统一、小部分区别对待的办法。例如，对每条指令都有的取指令操作，采用同步方式控制；对那些时间难以确定的微操作，如 I/O 操作，则采用异步控制。以执行部件送回的“回答”信号作为本次微操作的结束。